INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040
SYSTEM PROGRAMMING

WEEK 3 LECTURE 2

DATA REPRESENTATION

OFFLINE CLASS RULES

TO BE FOLLOWED VERY STRICTLY

1. BE PRESENT IN THE CLASS ATLEAST 5 MINUTES BEFORE THE START OF THE CLASSES
2. STUDENTS LATE BY MORE THAN 5 MINUTES WILL NOT BE ALLOWED TO ENTER THE CLASS
3. USE OF CELLPHONES & LAPTOPS STRICTLY PROHIBITED INSIDE THE CLASS
4. CELL PHONES & LAPTOPS SHOULD NOT BE KEPT ON THE DESK
5. CELLPHONES SHOULD BE KEPT EITHER IN YOUR POCKET OR BACKPACK/BAG YOU CARRY
6. SLEEPING IN CLASS IS TOTALLY PROHIBITED
7. BRING A NOTE BOOK FOR THIS COURSE AND MAKE NOTES IN THIS BOOK DURING THE CLASS
8. NO DRINKING & NO EATING IS ALLOWED IN CLASS (PLEASE NOTE THAT CLASSROOM IS NOT CANTEEN)

Lecture Outline

- Representing information as bits
  - Bit-level manipulations
- Integers
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
- Representations in memory, pointers, strings
- Real Number Representation
- Reading Assignment

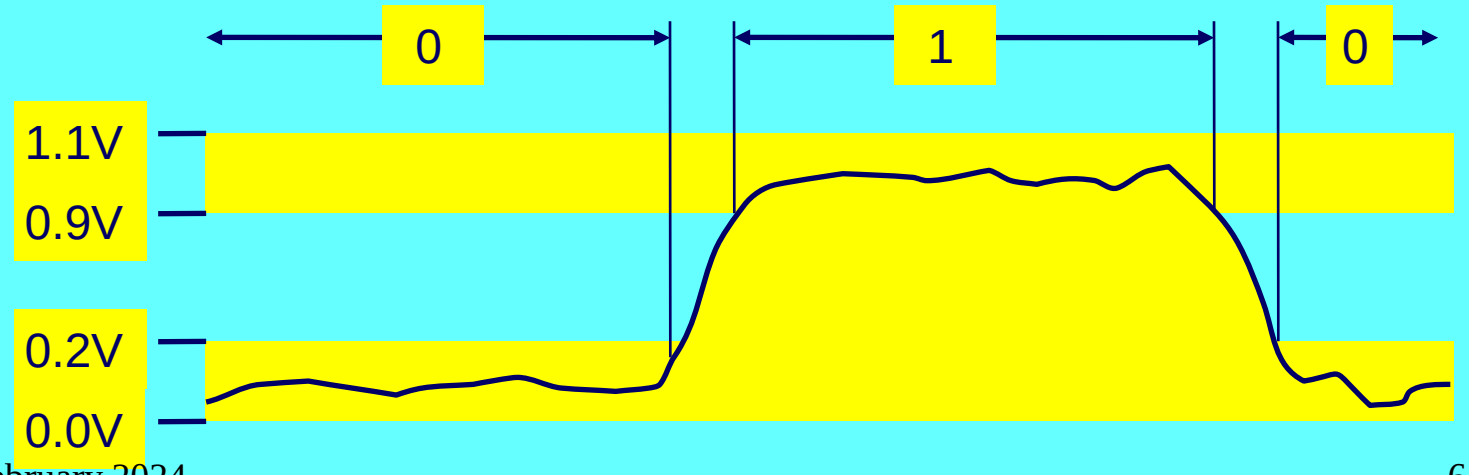
INFORMATION REPRESENTATION AS BITS

NUMBER SYSTEMS

BINARY, HEXADECIMAL & OCTAL

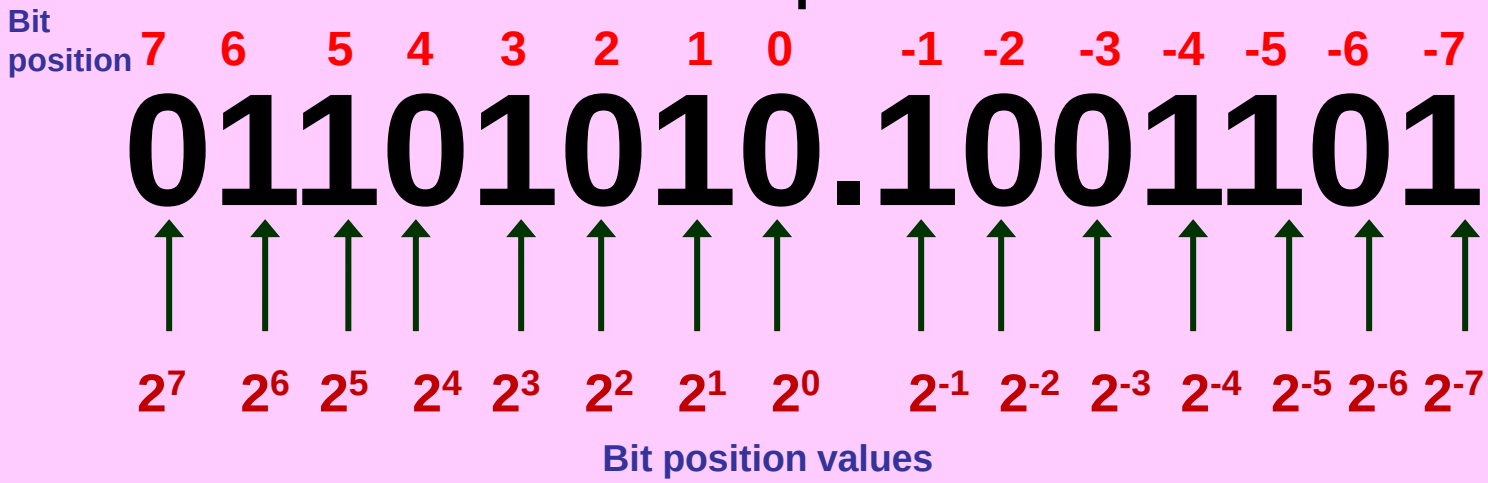
Everything is bits

- Each bit is 0 or 1
- By encoding/interpreting sets of bits in various ways
  - Computers determine what to do (instructions)
  - … and represent and manipulate numbers, sets, strings, etc…
- Why bits? Electronic Implementation
  - Easy to store with bistable elements
  - Reliably transmitted on noisy and inaccurate wires



Binary Representation

- Base 2 Number Representation



$= 0x\ 2^7 + 1 x\ 2^6 + 1 x\ 2^5 + 0 x\ 2^4 + 1 x\ 2^3 + 0x\ 2^2 + 1 x\ 2^1 + 0 x\ 2^0 + 1 x\ 2^{-1} + 0 x\ 2^{-2} + 0 x\ 2^{-3} + 1 x\ 2^{-4} + 1 x\ 2^{-5} + 0 x\ 2^{-6} + 1 x\ 2^{-7}$

= 0 + 64 + 32 + 0 + 8 + 0 + 2 + 0 + 0.5 + 0 + 0 + 0.0625+ 0.03125 + 0 + 0.0078125

=106.6015625

Convert Binary to Decimal

11011010.10101011

$= 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 0 \times 2^{-4} + 1 \times 2^{-5} + 0 \times 2^{-6} + 1 \times 2^{-7} + 1 \times 2^{-8}$

$= + 128 + 64 + 0 + 16 + 8 + 0 + 2 + 0 + 0.5 + 0 + 0.125 + 0 + 0.03125 + 0 + 0.0078125 + 0.00390625$

= 218.66796875

1011101011011010101.1011011011110110

$= 1 \times 2^{19} + 0 \times 2^{18} + 1 \times 2^{17} + 1 \times 2^{16} + 1 \times 2^{15} + 0 \times 2^{14} + 1 \times 2^{13} + 0 \times 2^{12} + 1 \times 2^{11} + 1 \times 2^{10} + 0 \times 2^9 + 1 \times 2^8 + 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 0 \times 2^{-5} + 1 \times 2^{-6} + 1 \times 2^{-7} + 0 \times 2^{-8} + 1 \times 2^{-9} + 1 \times 2^{-10} + 1 \times 2^{-11} + 1 \times 2^{-12} + 0 \times 2^{-13} + 1 \times 2^{-14} + 1 \times 2^{-15} + 0 \times 2^{-16}$

$= 5242288 + 0 + 131072 + 65536 + 32768 + 0 + 8192 + 0 + 2048 + 1024 + 0 + 256 + 128 + 0 + 32 + 16 + 0 + 4 + 0 + 1 + 0.5 + 0 + 0.125 + 0.0625 + 0 + .015625 + 0.0078125 + 0 + 0.001953125 + 0.0009765625 + 0.00048828125 + 0.000244140625 + 0.0001220703125 + 0.00006103515625 + 0.000030517578125 + 0.0000152587890625$

= 43702.7460479736328125

Convert Decimal 106.6015625 to Binary

2|106 .	
2| 53 - 0	
2| 26 - 1	
2| 13 - 0	
2| 6 - 1	
2| 3 - 0	
2| 1 - 1	
2| 0 - 1	

= 1101010

0.6015625 x 2	
1.2031250 x 2	- 1
0.4062500 x 2	- 0
0.8125000 x 2	- 0
1.6250000 x 2	- 1
1.2500000 x 2	- 1
0.5000000 x 2	- 0
1.0000000 x 2	- 1

= .1001101

106.6015625 = 1101010.1001101

Convert Decimal 985.78357 to Binary

2|985 .	
2|492 - 1	
2|246 - 0	
2|123 - 0	
2| 61 - 1	
2| 30 - 1	
2| 15 - 0	
2| 7 - 1	
2| 3 - 1	
2| 1 - 1	
2| 0 - 1	

= 1111011001

0.78357 x 2	
1.56714 x 2	- 1
1.13428 x 2	- 1
0.26856 x 2	- 0
0.53712 x 2	- 0
1.07424 x 2	- 1
0.14848 x 2	- 0
0.29696 x 2	- 0
0.59392 x 2	- 0
1.18784 x 2	- 1
0.37568 x 2	- 0
0.75136 x 2	- 0
1.50272 x 2	- 1
1.00544 x 2	- 1
0.01088 x 2	- 0
0.02176 x 2	- 0
0.04352 x 2	- 0

= .1100100010011000

985.78357= 1111011001.1100100010011000

Binary Representation

- **Base 2 Number Representation**
  - **Represent 15213_{10} as 11101101101101_2**
  - **Represent 1.20_{10} as $1.00110011001100 11[0011]\ldots_2$**
  - **Represent 1.5213×10^4 as $1.1101101101101_2 \times 2^{13}$**

Determine x in $10^4 = 2^x$

$10^4 = 2^x$

$\log_2 10^4 = \log_2 2^x$

$4 \log_2 10 = x \log_2 2$ where $\log_2 2 = 1$

$4 * 3.3219 = x$ where $\log_2 10 = 3.32192809489$

x = 13

Or **$10^4 = 2^{13}$**

Determine x in $10^x = 2^{13}$

$10^x = 2^{13}$

$\log_{10} 10^x = \log_{10} 2^{13}$

$x \log_{10} 10 = 13 \log_{10} 2$ where $\log_{10} 10 = 1$

$x = 13*0.301$ where $\log_{10} 2 = 0.301029995664$

x = 4

Or **$10^4 = 2^{13}$**

Hexadecimal Number System

- **Hexadecimal Numbers :**

  0,1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

- **Each hexadecimal digit corresponds to four binary digits**

Hex Number	Binary coded hex
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Example : Binary $(10110101)_2$ is Hex $(B5)_{16}$

Hexadecimal Number System

- **Convert decimal 215 to Hex**

- $(215)_{10} = (D7)_{16}$

16|215 .
16| 13 - 7
16| 0 - D

- **Convert Hex D7 to Binary**

- $(D7)_{16} = (11010111)_2$

- **Convert Hex D7 to Decimal**

- $(D7)_{16} = D \times 16^1 + 7 \times 16^0$

  $= 13 \times 16 + 7 \times 1 = (215)_{10}$

Encoding Byte Values

- Byte = 8 bits
  - **Binary 00000000_2 to 11111111_2**
  - **Decimal: 0_{10} to 255_{10}**
  - **Hexadecimal 00_{16} to FF_{16}**
    - Base 16 number representation
    - Use characters ‘0’ to ‘9’ and ‘A’ to ‘F’
    - Write $FA1D37B_{16}$ in C as
      - 0xFA1D37B
      - 0xfa1d37b

      a = 0xfa1d37b; b=100; decimal

      b=0x100; hexadecimal = 256 decimal

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Octal Number System

- **Octal Numbers : 0,1, 2, 3, 4, 5, 6, 7**
- **Each octal digit corresponds to three binary digits**

Octal Number	Binary coded Octal
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example : Binary $(100111101)_2$ is octal $(475)_8$

Octal Number System

- **Convert decimal 215 to Octal**

$8|\underline{215\quad .}$
$8|\underline{\ 26\ -\ 7}$
$8|\underline{\ \ 3\ -\ 2}$
$8|\underline{\ \ 0\ -\ 3}$

- $(215)_{10} = \quad (327)_8$

- **Convert octal 327 to Binary**

- $(327)_8 = (011010111)_2$

- **Convert octal 327 to Decimal**

- $(327)_8 = 3 \times 8^2 + 2 \times 8^1 + 7 \times 8^0$
  $= 192 + 16 + 7 = (215)_{10}$

Convert 0x879CEF to (i) Binary (ii) Decimal & (iii) Octal

(i) HEX2BIN(0x879CEF) = (10000111100111100 11101111)$_2$

(ii) HEX2DEC(0x879CEF)

$= 8 \times 16^5 + 7 \times 16^4 + 9 \times 16^3 + C \times 16^2 + E \times 16^1 + F \times 16^0$

$= 8 \times 1048576 + 7 \times 65536 + 9 \times 4096 + 12 \times 256 + 14 \times 16 + 15 \times 1$

$= 8388608 + 458752 + 36864 + 3072 + 224 + 15$

$= 8887535$

(iii) HEX2OCT(0x879CEF) = 100001111001110011101111 = (41716357)$_8$

Convert 0x4BAD6CAB to (i) Binary (ii) Decimal & (iii) Octal

(i) HEX2BIN(0x4BAD6CAB) = 01001011101011010110110010101011

(ii) HEX2DEC(0x4BAD6CAB)

$= 4 \times 16^7 + B \times 16^6 + A \times 16^5 + D \times 16^4 + 6 \times 16^3 + C \times 16^2 + A \times 16^1 + B \times 16^0$

$= 4 \times 268435456 + 11 \times 16777216 + 10 \times 1048576 + 13 \times 65536 + 6 \times 4096 + 12 \times 256 + 10 \times 16 + 11 \times 1$

$= +1073741824 + 184549376 + 10485760 + 851968 + 24576 + 3072 + 160 + 11$

$= 1269656747$

(iii) HEX2OCT(0x4BAD6CAB)

= 01001011101011010110110010101011= (11353266253)$_8$

Convert Octal 36745412 to (i) Binary (ii) Decimal & (iii) Hexadecimal

(i)OCT2BIN(36745412) = 011110111100101100001010

(ii)OCT2DEC(36745412)

$= 3 \times 8^7 + 6 \times 8^6 + 7 \times 8^5 + 4 \times 8^4 + 5 \times 8^3 + 4 \times 8^2 + 1 \times 8^1 + 2 \times 8^0$

$= 3 \times 2097152 + 6 \times 262144 + 7 \times 32768 + 4 \times 4096 + 5 \times 512 + 4 \times 64 + 1 \times 8 + 2$

$= 6291456 + 1572864 + 229376 + 16384 + 2560 + 256 + 10$

$= 8112906$

(iii) OCT2HEX(36745412) = 0111 1011 1100 1011 0000 1010= 0x7BCB0A

Convert Octal 753642532452 to (i) Binary (ii) Decimal & (iii) Hexadecimal

(i) OCT2BIN(753642532452) = 111101011110100010101011010100101010

(i) OCT2DEC(753642532452)

$= 7 \times 8^{11} + 5 \times 8^{10} + 3 \times 8^9 + 6 \times 8^8 + 4 \times 8^7 + 2 \times 8^6 + 5 \times 8^5 + 3 \times 8^4 + 2 \times 8^3 + 4 \times 8^2 + 5 \times 8^1 + 2 \times 8^0$

$= 7*8589934592 + + 5*1073741824 + 3* 134217728 + 6 \times 16777216 + 4 \times 2097152 + 2 \times 262144 + 5 \times 32768 + 3 \times 4096 + 2 \times 512 + 4 \times 64 + 5 \times 8 + 2$

$= 60129542144 + 5368709120 + 402653184 + 100663296 + 8388608 + 524288 + 163840 + 12288 + 1024 + 256 + 40 + 2$

$= 66010658090$

(i) OCT2HEX(753642532452)

= 111101011110100010101011010100101010 = 0xF5E8AB52A

Data Representations

DATA SIZES (in Bytes)

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
`char`	1	1	1
`short`	2	2	2
`int`	4	4	4
`long`	4	8	8
`float`	4	4	4
`double`	8	8	8
`long double`	–	–	10/16
pointer	4	8	8

SOC 2040
SYSTEM PROGRAMMING

BIT LEVEL MANIPULATIONS

Bit Level Manipulations

- **Boolean Algebra - Developed by George Boole in 19th Century**
  - **Algebraic representation of logic**
    - **Encode "True" as 1 and "False" as 0**

Bitwise and &

- **A&B = 1 when both A=1 and B=1**

&	0	1
0	0	0
1	0	1

Bitwise Or |

- **A|B = 1 when either A=1 or B=1**

|	0	1
0	0	1
1	1	1

Bitwise not ~

- **~A = 1 when A=0**

~	
0	1
1	0

Bitwise Exclusive-Or (Xor) ^

- **A^B = 1 when either A=1 or B=1, but not both**

^	0	1
0	0	1
1	1	0

General Boolean Algebras

- Operate on Bit Vectors
  - Operations applied bitwise

```
  01101001     01101001     01101001
& 01010101   | 01010101   ^ 01010101   ~ 01010101
----------   ----------   ----------   ----------
  01000001     01111101     00111100     10101010
```

- All of the Properties of Boolean Algebra Apply

Example: Representing & Manipulating Sets

- Representation
  - Width w bit vector represents subsets of {0, …, w–1}
  - $x_j = 1$ if $j \in X$
    - 01101001 { 0, 3, 5, 6 }
    - *76543210*
    - 01010101 { 0, 2, 4, 6 }
    - *76543210*
- Operations

– &	Intersection	01000001	{ 0, 6 }
– |	Union	01111101	{ 0, 2, 3, 4, 5, 6 }
– ^	Symmetric difference	00111100	{ 2, 3, 4, 5 }
– ~	Complement of 01010101	10101010	{ 1, 3, 5, 7 }

What is X^X ?

Bit-Level Operations in C

- **Operations &, |, ~, ^ Available in C**
  - Apply to any "integral" data type
    - **long, int, short, char, unsigned**
  - View arguments as bit vectors
  - Arguments applied bit-wise
- **Examples (Char data type)**

  ~0x41 → 0xBE (HEX)

  ~01000001_2 → 10111110_2 (BINARY)

  ~0x00 → 0xFF (HEX)

  ~00000000_2 → 11111111_2 (BINARY)

  0x69 & 0x55 → 0x41 (HEX)

  01101001_2 & 01010101_2 → 01000001_2 (BINARY)

  0x69 | 0x55 → 0x7D (HEX)

  01101001_2 | 01010101_2 → 01111101_2 (BINARY)

Contrast: Logic Operations in C

- Contrast to Logical Operators
  - &&, ||, !
    - View 0 as “False”
    - Anything nonzero as “True”
    - Always return 0 or 1
    - Early termination
- Examples (char data type)
  - !0x41 → 0x00
  - !0x00 → 0x01
  - !!0x41 → 0x01
  - 0x69 && 0x55 → 0x01
  - 0x69 || 0x55 → 0x01
  - p && *p (avoids null pointer access)

Contrast: Logic Operations in C

- Contrast to Logical Operators
  - &&, ||, !
    - Vi… 0 as "False"
    - Anyt… …zero as "True"
    - Alway…

Watch out for && vs. & (and || vs. |)… one of the more common oopsies in C programming

- …
  - 0x69 || 0x55 ➞ 0x01
  - p && *p (avoids null pointer access)

Contrast: Logic Operations in C

Given x = 0x4C and y = 0xB3, Compute the following showing all steps

(i) x && y = **(ii) x & y =**

(i) x && y

= 0x4C && 0xB3

= 0x01 && 0x01

= true && true

= true

= 0x01

= 1

Where true = 0x01 = 1

false = 0x00 = 0

(ii) x & y

= 0x4C & 0xB3

= 01001100 & 10110011

= 00000000

= false

= 0x00

= 0

```
  01001100
& 10110011
  00000000
```

Where true = 0x01 = 1

false = 0x00 = 0

SOC 2040
SYSTEM PROGRAMMING

END OF

WEEK 3 LECTURE2

DATA REPRESENTATION

WISH YOU ALL THE BEST

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 2 of Text Book

- **Computer Systems : A Programmer's Perspective**

  - **Randal E. Bryant and David R. O'Hallaron**
    3rd Edition, Global Edition Prentice Hall 2016
    ISBN 10:1-292-10176-8

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG